



PROJETO E ANÁLISE DE ALGORITMOS

AULA 6 – RECORRÊNCIAS

6.1 DIVISÃO E CONQUISTA

- Para o paradigma da divisão e conquista, resolvemos um problema (instância) recursivamente. Se o problema for pequeno o suficiente — o **caso-base** —, basta resolvê-lo diretamente, sem qualquer recursão. Caso contrário — o **caso recursivo** —, realizamos três etapas características da recursão:
 - **Divisão** do problema em certo número de subproblemas que são instâncias menores do mesmo problema.
 - **Conquista** dos subproblemas, resolvendo-os recursivamente.
 - **Combinação** das soluções dos subproblemas para formar a solução do problema original.

6.1 DIVISÃO E CONQUISTA

- Um algoritmo de divisão e conquista desmembra um problema grande em **subproblemas menores**, que, por sua vez, podem ser desmembrados em subproblemas ainda menores, e assim por diante.
- Dizemos que a recursão **se esgota** quando alcança o caso-base e o subproblema é pequeno o suficiente para ser resolvido diretamente, sem mais recursão.

6.2 RECORRÊNCIAS

- Uma recorrência é uma **equação** que descreve uma função em termos de seu valor em outros argumentos, normalmente menores.
- As recorrências andam de mãos dadas com o método de divisão e conquista porque nos dão uma maneira natural de caracterizar matematicamente os tempos de execução de **algoritmos recursivos**.
- A forma geral de uma recorrência é uma equação ou inequação que descreve uma função sobre os valores inteiros ou reais **usando a própria função**. Ela contém dois ou mais casos, dependendo do argumento.

6.2 RECORRÊNCIAS

- Uma **recorrência** $T(n)$ é **algorítmica** se, para cada constante de limiar suficientemente grande $n_0 > 0$, as seguintes propriedades forem verdadeiras:
 - Para todo $n < n_0$, temos $T(n) = \Theta(1)$.
 - Para todo $n \geq n_0$, cada caminho de recursão termina em um caso-base dentro de um número **finito** de chamadas recursivas.

6.2 RECORRÊNCIAS

- Adotamos a seguinte convenção:

Sempre que uma recorrência é enunciada sem um caso-base explícito, consideramos que a recorrência é algorítmica.

- Isso significa que você pode escolher qualquer constante de limiar n_0 suficientemente grande para o intervalo de casos base em que $T(n) = \Theta(1)$. As soluções assintóticas da maioria das recorrências algorítmicas geralmente não dependem da escolha da constante de limiar, desde que seja grande o suficiente para tornar a recorrência bem definida.

6.2 RECORRÊNCIAS

- Ocasionalmente, veremos recorrências que **não são equações**, mas sim inequações, como $T(n) \leq 2T(n/2) + \Theta(n)$.
- Como tal recorrência declara somente um **limite superior** para $T(n)$, expressaremos sua solução usando a notação O em vez da notação Θ .
- De modo semelhante, se a desigualdade for invertida para $T(n) \geq 2T(n/2) + \Theta(n)$, então, como a recorrência dá apenas um **limite inferior** para $T(n)$, usaremos a notação Ω em sua solução.

6.2 RECORRÊNCIAS

- Embora seja comum, ao se usar a divisão e conquista, que todos os subproblemas tenham o **mesmo tamanho**, nem sempre isso acontece. Por vezes, é produtivo dividir um problema de tamanho n em subproblemas de tamanhos diferentes e, então, a recorrência que descreve o tempo de execução reflete a irregularidade.
- Embora os algoritmos de divisão e conquista geralmente criem subproblemas com os tamanhos de uma **fração constante** do tamanho original do problema, isso nem sempre acontece.

6.2 RECORRÊNCIAS

- Uma recorrência pode ser vista como um algoritmo recursivo que calcula uma função a partir de um valor inicial.
- Uma recorrência é satisfeita por muitas funções diferentes, uma para cada valor inicial; mas todas essas funções são, em geral, do mesmo tipo. As funções que nos interessam são, via de regra, definidas no conjunto dos números naturais. Mas podem ser definida em outros conjuntos, como os naturais maiores que 99, as potências inteiras de 2, as potências inteiras de $1\frac{1}{2}$, os racionais maiores que $\frac{1}{2}$, etc.

6.2 RECORRÊNCIAS

- Vamos nos concentrar em 4 métodos para resolver recorrências:
 - Método iterativo: expandimos (ou “desenrolamos”) a recorrência até chegarmos no caso base, que sabemos resolver diretamente.
 - Método de substituição: arriscamos um palpite para a forma de um limite e então usamos indução matemática para provar que nosso palpite estava correto e resolver as constantes. Esse método talvez seja o mais robusto para resolver recorrências, mas também exige que se faça uma boa suposição e produza uma prova indutiva.

6.2 RECORRÊNCIAS

- Vamos nos concentrar em 4 métodos para resolver recorrências:
 - Método da árvore de recursão: converte a recorrência em uma árvore cujos nós representam os custos envolvidos em vários níveis da recursão. Para resolver a recorrência, você determina os custos em cada nível e os soma, talvez usando técnicas para limitar somatórios. Mesmo que não se use esse método para provar um limite de modo formal, ele pode ser útil para adivinhar a forma do limite para uso no método de substituição. Dessa forma, os dois métodos podem ser combinados.

6.2 RECORRÊNCIAS

- Vamos nos concentrar em 4 métodos para resolver recorrências:
 - O método (ou teorema) mestre: é o mais fácil, quando aplicável. Ele dá limites para recorrências da forma:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

em que $a > 0$ e $b > 1$ são constantes e $f(n)$ é uma função de “direção” dada. Esse tipo de recorrência é o mais frequente e caracteriza um algoritmo de divisão e conquista que cria subproblemas, cada um com $1/b$ do tamanho do problema original e no qual as etapas de divisão e conquista, juntas, levam o tempo $f(n)$.

6.2 RECORRÊNCIAS

- Resolver uma recorrência é encontrar uma fórmula fechada que dê o valor da função diretamente em termos de seu argumento (e sem sub-expressões da forma $+$... $+$ ou contendo $\sum$ ou $\prod$).
- Tipicamente, a fórmula fechada é uma combinação de polinômios, quocientes de polinômios, logaritmos, exponenciais, etc.

6.2 RECORRÊNCIAS

■ Exemplo:

$$\text{■ } T(n) = \begin{cases} 3 & \text{se } n = 0 \\ 5 & \text{se } n = 1 \\ 2T(n-1) - T(n-2) & \text{se } n > 1 \end{cases}$$

6.2 RECORRÊNCIAS

■ Exemplo:

$$■ T(n) = \begin{cases} 3 & \text{se } n = 0 \\ 5 & \text{se } n = 1 \\ 2T(n-1) - T(n-2) & \text{se } n > 1 \end{cases}$$

■ $T(0)=3$

■ $T(1)=5$

■ $T(2)=7$

■ $T(3)=9$

Parece que:

$$T(n)=2n+3 \quad \forall n \geq 0$$

6.2 RECORRÊNCIAS

■ Exemplo:

$$■ T(n) = \begin{cases} 3 & \text{se } n = 0 \\ 5 & \text{se } n = 1 \\ 2T(n-1) - T(n-2) & \text{se } n > 1 \end{cases}$$

■ $T(0)=3$

■ $T(1)=5$

■ $T(2)=7$

■ $T(3)=9$

Por indução:

$$T(0)=2 \cdot 0 + 3 = 3$$

$$T(1)=2 \cdot 1 + 5 = 5$$

(ok)

6.2 RECORRÊNCIAS

■ Exemplo:

$$■ T(n) = \begin{cases} 3 & \text{se } n = 0 \\ 5 & \text{se } n = 1 \\ 2T(n-1) - T(n-2) & \text{se } n > 1 \end{cases}$$

■ $T(0)=3$

■ $T(1)=5$

■ $T(2)=7$

■ $T(3)=9$

Por indução:

Se $T(n) = 2n + 3$ é verdade para $n=k-1$ e $n=k-2$, então:

$$T(k-1) = 2(k-1) + 3 = 2k + 1$$

$$T(k-2) = 2(k-2) + 3 = 2k - 1$$

6.2 RECORRÊNCIAS

■ Exemplo:

$$■ T(n) = \begin{cases} 3 & \text{se } n = 0 \\ 5 & \text{se } n = 1 \\ 2T(n-1) - T(n-2) & \text{se } n > 1 \end{cases}$$

■ $T(0)=3$

■ $T(1)=5$

■ $T(2)=7$

■ $T(3)=9$

Por indução:

Se $T(n) = 2n + 3$ é verdade para $n=k-1$ e $n=k-2$, então:

$$T(k-1) = 2(k-1) + 3 = 2k + 1$$

$$T(k-2) = 2(k-2) + 3 = 2k - 1$$

Pela recorrência:

$$T(k) = 2T(k-1) - T(k-2)$$

Que, pela hipótese é:

$$T(k) = 2(2k+1) - (2k-1)$$

$$T(k) = 4k+2-2k+1 = \mathbf{2k+3}$$

6.2 RECORRÊNCIAS

■ Exemplo:

$$■ T(n) = \begin{cases} 3 & \text{se } n = 0 \\ 5 & \text{se } n = 1 \\ 2T(n-1) - T(n-2) & \text{se } n > 1 \end{cases}$$

■ $T(0)=3$

■ $T(1)=5$

■ $T(2)=7$

■ $T(3)=9$

De fato:

$$T(n)=2n+3 \quad \forall n \geq 0$$

Por indução:

Se $T(n) = 2n + 3$ é verdade para $n=k-1$ e $n=k-2$, então:

$$T(k-1) = 2(k-1)+3 = 2k + 1$$

$$T(k-2) = 2(k-2)+3 = 2k - 1$$

Pela recorrência:

$$T(k) = 2T(k-1) - T(k-2)$$

Que, pela hipótese é:

$$T(k) = 2(2k+1) - (2k-1)$$

$$T(k) = 4k+2-2k+1 = \mathbf{2k+3}$$

6.3 MÉTODO ITERATIVO

- Considere o algoritmo:

SOMPOSITIVOS (A, n)

1 se $n = 0$

2 então devolva 0

3 senão $s \leftarrow \text{SOMPOSITIVOS}(A, n - 1)$

4 se $A[n] > 0$

5 então devolva $s + A[n]$

6 senão devolva s

6.3 MÉTODO ITERATIVO

- Seja $T(n)$ a função que dá o consumo de tempo no pior caso.
- Se a execução de qualquer das linhas do pseudocódigo consome uma unidade de tempo, então $T(0) = 2$ e $T(n)$ satisfaz a recorrência

$$T(n) = T(n - 1) + 3$$

- O termo $T(n-1)$ corresponde à linha 3 e o termo 3 é o consumo de tempo das linhas 1 e linhas 4–6.
- Assim, $T(1) = 2 + 3 = 5$, $T(2) = 5 + 3 = 8$, $T(3) = 8 + 3 = 11$, etc.

6.3 MÉTODO ITERATIVO

- Aplicando o método iterativo:

$$\begin{aligned}T(n) &= T(n - 1) + 3 \\&= (T(n - 2) + 3) + 3 \\&= T(n - 2) + 6 \\&= T(n - 3) + 9 \\&= \dots \\&= T(n - i) + 3i \\&= T(0) + 3n \\&= 2 + 3n.\end{aligned}$$

6.3 MÉTODO ITERATIVO

- Exemplo 2:

$$T(n) = T(n - 1) + n + 1, T(0) = 1$$

6.3 MÉTODO ITERATIVO

- Exemplo 2:

$$T(n) = T(n - 1) + n + 1, T(0) = 1$$

$$T(n) = T(n - 1) + n + 1 = T(n - 2) + (n - 1) + 1 + n + 1$$

6.3 MÉTODO ITERATIVO

- Exemplo 2:

$$T(n) = T(n - 1) + n + 1, T(0) = 1$$

$$\begin{aligned} T(n) &= T(n - 1) + n + 1 = T(n - 2) + (n - 1) + 1 + n + 1 \\ &= T(n - 2) + (n - 1) + n + 2 \end{aligned}$$

6.3 MÉTODO ITERATIVO

- Exemplo 2:

$$T(n) = T(n - 1) + n + 1, T(0) = 1$$

$$T(n) = T(n - 1) + n + 1 = T(n - 2) + (n - 1) + 1 + n + 1$$

$$= T(n - 2) + (n - 1) + n + 2$$

$$= T(n - 3) + (n - 2) + 1 + (n - 1) + n + 2$$

$$= T(n - 3) + (n - 2) + (n - 1) + n + 3$$

$$= T(n - 4) + (n - 3) + (n - 2) + (n - 1) + n + 4$$

6.3 MÉTODO ITERATIVO

■ Exemplo 2:

$$T(n) = T(n - 1) + n + 1, T(0) = 1$$

$$T(n) = T(n - 1) + n + 1 = T(n - 2) + (n - 1) + 1 + n + 1$$

$$= T(n - 2) + (n - 1) + n + 2$$

$$= T(n - 3) + (n - 2) + 1 + (n - 1) + n + 2$$

$$= T(n - 3) + (n - 2) + (n - 1) + n + 3$$

$$= T(n - 4) + (n - 3) + (n - 2) + (n - 1) + n + 4$$

$$= T(n - i) + (n - i + 1) + (n - i + 2) + \cdots + (n - 1) + n + i$$

6.3 MÉTODO ITERATIVO

- O processo de expandir a recorrência revelou um padrão. Esse padrão permite concluir que $F(n) = F(0) + 1 + 2 + \dots + (n-2) + (n-1) + n + n$. Se o valor inicial é $F(0) = 1$ então
- $F(n) = 1 + n(n+1)/2 + n = \frac{1}{2} (n^2 + 3n + 2)$.

6.3 MÉTODO ITERATIVO

- Exercício:

$$T(n) = T\left(\frac{n}{2}\right) + 3$$

6.3 MÉTODO ITERATIVO

- Exercício:

$$T(n) = T\left(\frac{n}{2}\right) + 3$$

- Faz sentido que n pertença ao conjunto $\{2, 3, 4, 5, \dots\}$?

6.3 MÉTODO ITERATIVO

- Exercício:

$$T(n) = T\left(\frac{n}{2}\right) + 3$$

- Faz sentido que n pertença ao conjunto $\{2, 3, 4, 5, \dots\}$?
- Não porque $n/2$ não pertence a esse conjunto quando n é ímpar. Também não faz sentido tomar n no conjunto $\{2, 4, 6, 8, \dots\}$ pois $n/2$ não pertence a esse conjunto quando n não é o dobro de um número par. A recorrência faz sentido, entretanto, se n pertence ao conjunto $\{2^1, 2^2, 2^3, 2^4, \dots\}$ das potências inteiras de 2. Nesse caso, podemos reescrever a recorrência assim:

6.3 MÉTODO ITERATIVO

- Exercício:

$$T(n) = T\left(\frac{n}{2}\right) + 3$$

- Nesse caso, podemos reescrever a recorrência assim:

$$T(2^k) = T(2^{k-1}) + 3$$

6.3 MÉTODO ITERATIVO

- Exercício:

$$\begin{aligned}T(2^k) &= T(2^{k-1}) + 3 = T(2^{k-2}) + 3 + 3 = T(2^{k-3}) + 3 + 3 + 3 \\ &= T(2^{k-k}) + 3k = F(1) + 3k\end{aligned}$$

- Assumindo $T(1)=1$ e $k = \lg n$, temos:

$$T(n) = 3\lg n + 1$$

6.3 MÉTODO ITERATIVO

- Usamos o valor inicial $T(1) = 1$ pois os valores iniciais de uma recorrência — ou seja, os valores de $T(n)$ quando n é pequeno — não têm influência sobre a forma geral de $T(n)$. Podemos então adotar quaisquer valores convenientes.
- Em princípio, todas as relações de recorrência podem ser resolvidas aplicando o método iterativo. No entanto, nem sempre ele fornece uma solução simples, pois pode ser difícil lidar com as propriedades matemáticas necessárias para obter uma fórmula fechada para algumas recorrências.

6.3 MÉTODO ITERATIVO

- Exercício:
- $T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T\left(\left\lceil \frac{n}{2} \right\rceil\right) + n + 1$

6.3 MÉTODO ITERATIVO

- Exercício:
- $T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T\left(\left\lceil \frac{n}{2} \right\rceil\right) + n + 1$
- Observação: não precisamos de uma solução exata da recorrência. Ficaremos muito satisfeitos com uma fórmula assintótica, como $T(n) = O(n \lg n)$ por exemplo. O primeiro passo nessa direção é resolver uma recorrência semelhante **mais simples**.

6.3 MÉTODO ITERATIVO

- Exercício:

$$T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T\left(\left\lceil \frac{n}{2} \right\rceil\right) + n$$

- Restrição às potências de 2: a restrição da recorrência às potências de 2 tem a vantagem de eliminar os incômodos das funções piso e teto.
- Para $n = 2^1, 2^2, 2^3, \dots$, temos simplesmente:

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

6.3 MÉTODO ITERATIVO

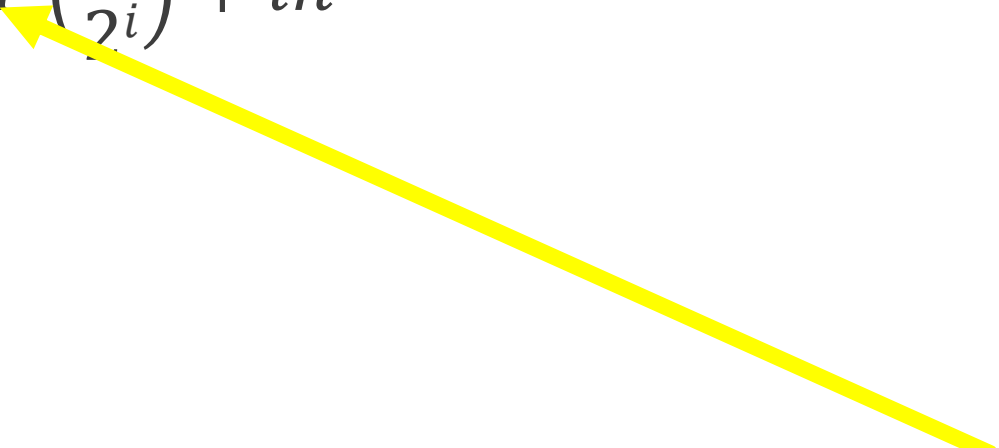
■ Exercício:

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + n = 2\left(2T\left(\frac{n}{4}\right) + \frac{n}{2}\right) + n = 2^2T\left(\frac{n}{2^2}\right) + 2n \\ &= 2^3T\left(\frac{n}{2^3}\right) + 3n = 2^iT\left(\frac{n}{2^i}\right) + in \end{aligned}$$

6.3 MÉTODO ITERATIVO

■ Exercício:

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + n = 2\left(2T\left(\frac{n}{4}\right) + \frac{n}{2}\right) + n = 2^2T\left(\frac{n}{2^2}\right) + 2n \\ &= 2^3T\left(\frac{n}{2^3}\right) + 3n = 2^iT\left(\frac{n}{2^i}\right) + in \end{aligned}$$


$$a^{\log_a b} = b$$

6.3 MÉTODO ITERATIVO

- Exercício:

$$\begin{aligned}T(n) &= 2T\left(\frac{n}{2}\right) + n = 2\left(2T\left(\frac{n}{4}\right) + \frac{n}{2}\right) + n = 2^2T\left(\frac{n}{2^2}\right) + 2n \\&= 2^3T\left(\frac{n}{2^3}\right) + 3n = 2^iT\left(\frac{n}{2^i}\right) + in \\&= nT\left(\frac{n}{n}\right) + n\lg n = n + n\lg n\end{aligned}$$

6.4 MÉTODO DE SUBSTITUIÇÃO

- O método de substituição envolve duas etapas:
 1. **Arriscar** um palpite para a forma da solução usando constantes simbólicas.
 2. Usar **indução matemática** para mostrar que a solução funciona e determinar as constantes.
- **Substituimos** a função pela solução suposta na primeira etapa quando aplicamos a hipótese indutiva a valores menores — daí o nome método de substituição.

6.4 MÉTODO DE SUBSTITUIÇÃO

- Podemos usar o método de substituição para estabelecer limites superiores ou inferiores para uma recorrência. Normalmente, é melhor **não tentar fazer ambos** ao mesmo tempo. Isto é, em vez de tentar provar um limite Θ diretamente, primeiro é melhor provar um limite O e depois um limite Ω . Juntos, eles lhe darão um limite Θ .
- Vamos usar o método da substituição para determinar um limite assintótico superior para a recorrência:

$$T(n) = 2T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + \theta(n)$$

6.4 MÉTODO DE SUBSTITUIÇÃO

- Essa recorrência é semelhante à da ordenação por intercalação, exceto para a função de piso, que garante que $T(n)$ é definida sobre os inteiros.
- Vamos supor que o limite assintótico superior seja o mesmo — $T(n) = O(n \lg n)$ — e usar o método da substituição para provar isso.
- Vamos adotar a hipótese indutiva de que $T(n) \leq cn \lg n$ para todo $n \geq n_0$, em que escolheremos as constantes específicas $c > 0$ e $n_0 > 0$ posteriormente, depois de vermos as restrições às quais elas precisam obedecer.

6.4 MÉTODO DE SUBSTITUIÇÃO

- Suponha, por indução, que esse limite seja válido para todos os números pelo menos tão grandes quanto n_0 e menores que n .
- Em particular, portanto, se $n \geq 2n_0$, ele se mantém para $\lfloor n/2 \rfloor$, o que produz $T(\lfloor n/2 \rfloor) \leq c \lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)$. Substituindo na recorrência obtemos:

$$T(n) \leq 2 \left(c \left\lfloor \frac{n}{2} \right\rfloor \lg \left(\left\lfloor \frac{n}{2} \right\rfloor \right) \right) + \theta(n)$$

6.4 MÉTODO DE SUBSTITUIÇÃO

$$\begin{aligned}T(n) &\leq 2 \left(c \left\lfloor \frac{n}{2} \right\rfloor \lg \left(\left\lfloor \frac{n}{2} \right\rfloor \right) \right) + \theta(n) \leq 2 \left(c \left(\frac{n}{2} \right) \lg \left(\frac{n}{2} \right) \right) + \theta(n) \\&= cn \lg \left(\frac{n}{2} \right) + \theta(n) = cn \lg n - cn \lg 2 + \theta(n) \\&= cn \lg n - cn + \theta(n) \leq cn \lg n\end{aligned}$$

- em que a última etapa é válida desde que restrinjamos as constantes n_0 e c a serem suficientemente grandes de modo que, para $n \geq 2n_0$, a quantidade cn domine a função anônima ocultada pelo termo $\Theta(n)$.

6.4 MÉTODO DE SUBSTITUIÇÃO

- Mostramos que a hipótese indutiva é válida para o caso indutivo, mas também precisamos provar que a hipótese indutiva vale para os casos-base da indução, ou seja, que $T(n) \leq cn \lg n$ quando $n_0 \leq n < 2n_0$.
- Lembre-se que $\log_a 1 = 0$. Enquanto $n_0 > 1$, temos $\lg n > 0$, o que implica que $n \lg n > 0$. Portanto, vamos escolher $n_0 = 2$.
- Como o caso-base da recorrência não é declarado explicitamente, por nossa convenção, $T(n)$ é algorítmico, o que significa que $T(1)$, $T(2)$ e $T(3)$ são constantes (como devem ser se descrevem o tempo de execução do pior caso de qualquer programa real sobre entradas de tamanho 1, 2 ou 3).

6.4 MÉTODO DE SUBSTITUIÇÃO

- Vamos supor, como argumento, que $T(1) = 1$ seja o único caso base da recorrência. Então, para $n = 1$, o limite $T(n) \leq cn \lg n$ produz $T(1) \leq c \cdot 1 \lg 1 = 0$, o que está em desacordo com $T(1) = 1$.
- Consequentemente, o caso-base de nossa prova indutiva deixa de ser válido.
- Para superar o problema, consideramos que a notação assintótica só exige que provemos que $T(n) \leq cn \lg n$ para $n \geq n_0$, em que n_0 é uma constante de **nossa escolha**.

6.4 MÉTODO DE SUBSTITUIÇÃO

- Mantemos a condição de contorno $T(1) = 1$, mas não a consideramos na prova indutiva. Fazemos isso primeiro observando que, para $n > 3$, a recorrência não depende diretamente de $T(1)$. Desse modo, podemos substituir $T(1)$ por $T(2)$ e $T(3)$ como os casos-base na prova indutiva fazendo $n_0 = 2$.
- $T(n)$ é algorítmico, o que significa que $T(2)$ e $T(3)$ são constantes (como devem ser se descrevem o tempo de execução do pior caso de qualquer programa real sobre entradas de tamanho 2 ou 3).

6.4 MÉTODO DE SUBSTITUIÇÃO

- Observe que fazemos uma distinção entre o caso base da recorrência ($n = 1$) e os casos-base da prova indutiva ($n = 2$ e $n = 3$). Com $T(1) = 1$, derivamos da recorrência que $T(2) = 4$ e $T(3) = 5$.
- Agora podemos concluir a prova indutiva de $T(n) \leq cn \lg n$ para alguma constante $c \geq 1$ escolhendo c suficientemente grande de modo que $T(2) \leq c \cdot 2 \cdot \lg 2$ e $T(3) \leq c \cdot 3 \cdot \lg 3$.
- Qualquer valor de $c \geq 2$ é suficiente para que os casos-base de $n = 2$ e $n = 3$ sejam válidos.

6.4 MÉTODO DE SUBSTITUIÇÃO

- Infelizmente, **não há nenhum modo geral** para adivinhar corretamente as soluções assintóticas mais restritas para recorrências arbitrárias. Arriscar um palpite para solução exige experiência e, ocasionalmente, criatividade. Entretanto, é possível usar árvores de recursão para gerar bons palpites.
- Se uma recorrência for semelhante a alguma que **já conhecida**, será razoável supor uma solução semelhante.

6.4 MÉTODO DE SUBSTITUIÇÃO

- Como exemplo, considere a recorrência:

$$T(n) = 2T\left(\frac{n}{2} + 17\right) + \theta(n)$$

- definida sobre valores reais. Essa recorrência se parece um pouco com a recorrência de ordenação por intercalação, porém é mais complicada em razão do “17” acrescentado ao argumento de T no lado direito. Porém, intuitivamente, esse termo adicional não pode afetar substancialmente a solução para a recorrência. Quando n é grande, a diferença entre $n/2$ e $n/2 + 17$ não é tão grande: ambos cortam n quase uniformemente pela metade. Em consequência disso, arriscamos $T(n) = O(n \lg n)$ e verificamos que é correto usando o método de substituição.

6.4 MÉTODO DE SUBSTITUIÇÃO

- Exercício: use o método da substituição para mostrar que cada uma das seguintes recorrências definida sobre valores reais tem a solução assintótica especificada:
 - a) $T(n) = T(n - 1) + n$ tem solução $T(n) = O(n^2)$.
 - b) $T(n) = T(n/2) + \Theta(1)$ tem solução $T(n) = O(\lg n)$.
 - c) $T(n) = 2T(n/2) + n$ tem solução $T(n) = \Theta(n \lg n)$.
 - d) $T(n) = 2T(n/2 + 17) + n$ tem solução $T(n) = O(n \lg n)$.
 - e) $T(n) = 2T(n/3) + \Theta(n)$ tem solução $T(n) = \Theta(n)$.
 - f) $T(n) = 4T(n/2) + \Theta(n)$ tem solução $T(n) = \Theta(n^2)$.

6.5 MÉTODO DA ÁRVORE DE RECURSÃO

- Em uma árvore de recursão, cada nó representa o custo de um único subproblema em algum lugar no conjunto de invocações da função recursiva.
- Somamos os custos em cada nível da árvore para obter um conjunto de custos por nível e depois somamos todos os custos por nível para determinar o custo total de todos os níveis da recursão.
- Porém, eventualmente, o somatório do custo total exige mais criatividade.

6.5 MÉTODO DA ÁRVORE DE RECURSÃO

- Uma árvore de recursão é mais bem usada para gerar um bom palpite, que é então verificado pelo método de substituição. No entanto, se você for muito cuidadoso ao desenhar uma árvore de recursão e somar os custos, poderá usar uma árvore de recursão como prova direta de solução para uma recorrência.
- Porém, se ela for usada somente para gerar um bom palpite, você pode tolerar um pouco de “desleixo”, o que pode simplificar a matemática. Ao verificar seu palpite com o método de substituição mais adiante, sua matemática deverá ser precisa.

6.5 MÉTODO DA ÁRVORE DE RECURSÃO

- Exemplo: limite superior para a recorrência

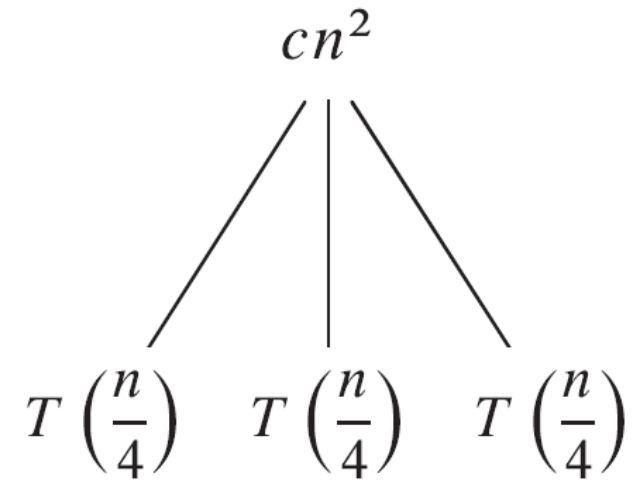
$$T(n) = 3T\left(\frac{n}{4}\right) + \theta(n^2)$$

6.5 MÉTODO DA ÁRVORE DE RECURSÃO

- Exemplo: limite superior para a recorrência

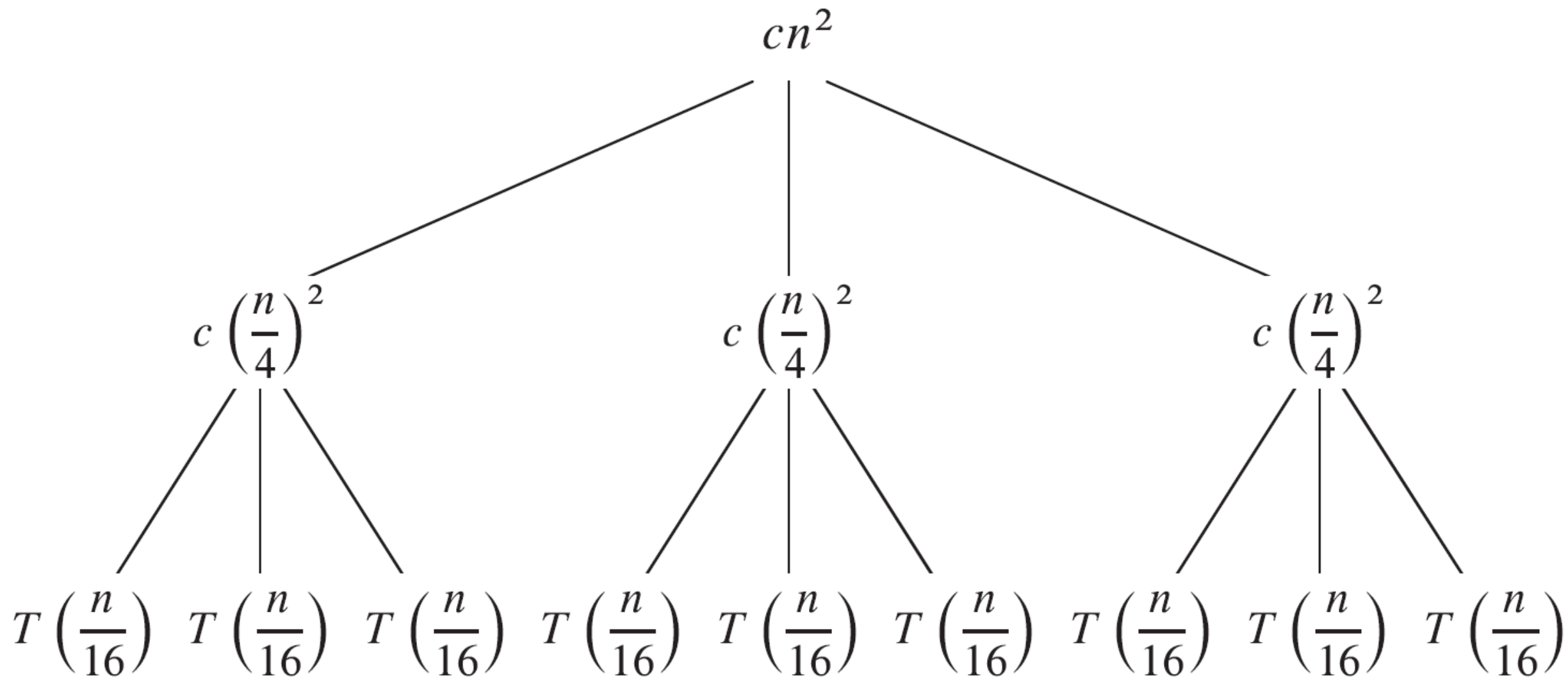
$$T(n) = 3T\left(\frac{n}{4}\right) + \theta(n^2)$$

$T(n)$



(a)

(b)



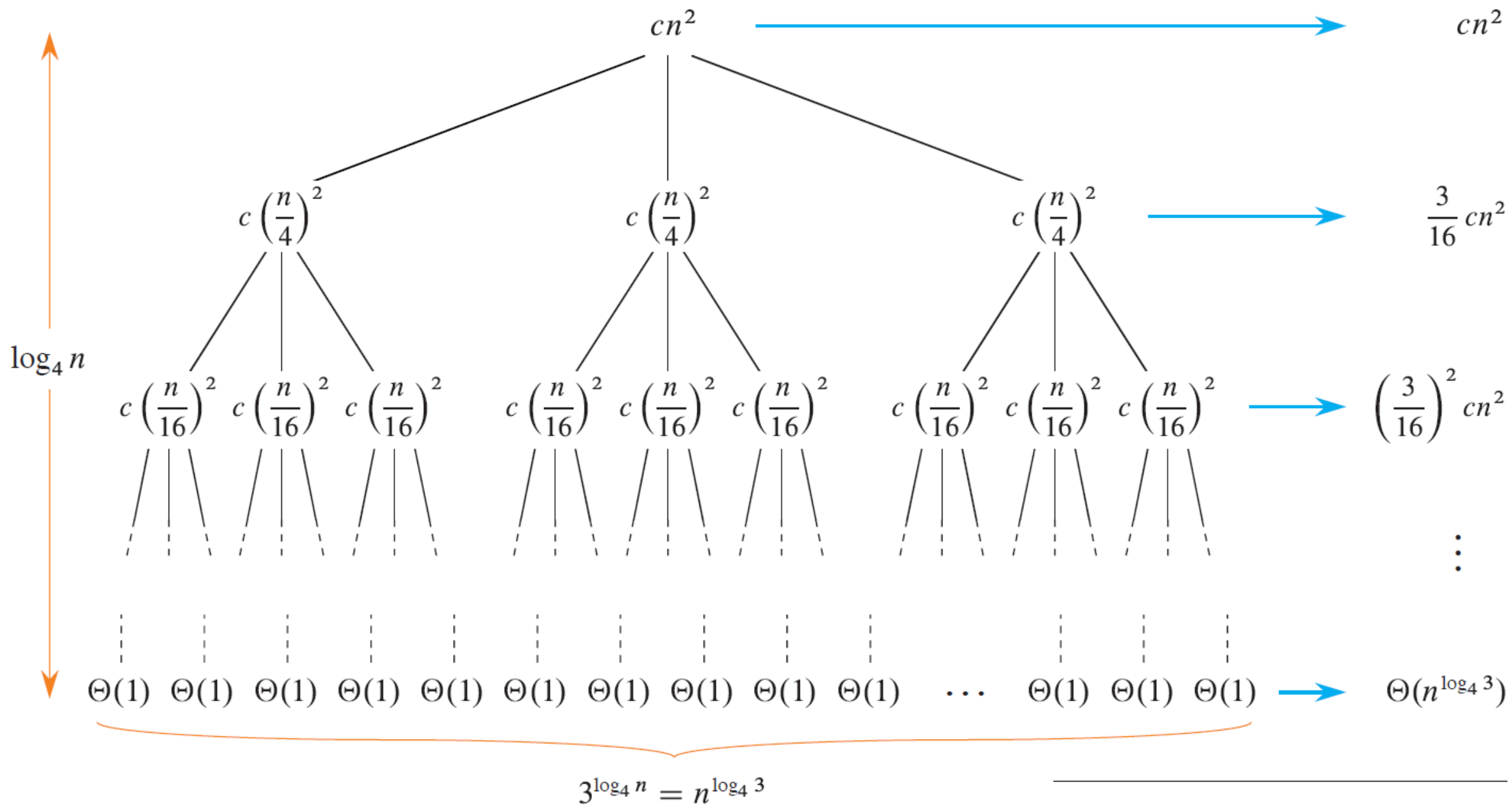
(c)

6.5 MÉTODO DA ÁRVORE DE RECURSÃO

- Visto que os tamanhos dos subproblemas diminuem por um fator de 4 toda vez que descemos um nível, a certa altura devemos alcançar um caso-base em que $n < n_0$. Por convenção, o caso-base é $T(n) = \Theta(1)$ para $n < n_0$, em que $n_0 > 0$ é qualquer constante de limiar suficientemente grande para que a recorrência seja bem definida.
- Contudo, por questão de intuição, vamos simplificar um pouco a matemática. Vamos supor que n seja uma potência exata de 4 e que o caso-base é $T(1) = \Theta(1)$. Sabemos que essas hipóteses **não afetam** a solução assintótica.

6.5 MÉTODO DA ÁRVORE DE RECURSÃO

- Qual é a altura da árvore de recursão?
- O tamanho do subproblema para um nó na profundidade i é $n/4^i$. Ao descermos a árvore a partir da raiz, o tamanho do subproblema chega a $n = 1$ quando $n/4^i = 1$ ou, o que é equivalente, quando $i = \log_4 n$.
- Assim, a árvore tem nós internos nas profundidades $0, 1, 2, \dots, \log_4 n - 1$ e folhas na profundidade $\log_4 n$.



(d)

Total: $O(n^2)$

6.5 MÉTODO DA ÁRVORE DE RECURSÃO

- Cada nível tem três vezes mais nós que o nível acima dele, portanto o número de nós na profundidade i é 3^i .
- Como os tamanhos dos subproblemas se reduzem por um fator de 4 para cada nível que descemos a partir da raiz, cada nó na profundidade i , para $i = 0, 1, 2, \dots, \log_4 n - 1$, tem o custo de $c(n/4^i)^2$. Multiplicando, vemos que o custo total para todos os nós na profundidade i é $3^i c(n/4^i)^2 = (3/16)^i c n^2$. O nível inferior, na profundidade $\log_4 n$, tem $3^{\log_4 n} = n^{\log_4 3}$ folhas (pois $a^{\log_b c} = c^{\log_b a}$).
- Cada folha contribui com $\Theta(1)$, levando a um custo total de $\Theta(n^{\log_4 3})$.

6.5 MÉTODO DA ÁRVORE DE RECURSÃO

- Agora somamos os custos em todos os níveis para determinar o custo da árvore inteira:

$$\begin{aligned} T(n) &= cn^2 + \frac{3}{16}cn^2 + \left(\frac{3}{16}\right)^2 cn^2 + \dots + \left(\frac{3}{16}\right)^{\log_4 n - 1} cn^2 + \Theta(n^{\log_4 3}) \\ &= \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\ &< \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\ &= \frac{1}{1 - (3/16)} cn^2 + \Theta(n^{\log_4 3}) \\ &= \frac{16}{13} cn^2 + \Theta(n^{\log_4 3}) \\ &= O(n^2) \quad (\Theta(n^{\log_4 3}) = O(n^{0,8}) = O(n^2)) . \end{aligned}$$

6.5 MÉTODO DA ÁRVORE DE RECURSÃO

- Agora somamos os custos em todos os níveis para determinar o custo da árvore inteira:

$$T(n) = cn^2 + \frac{3}{16}cn^2 + \left(\frac{3}{16}\right)^2 cn^2 + \dots + \left(\frac{3}{16}\right)^{\log_4 n - 1} cn^2 + \Theta(n^{\log_4 3})$$

$$= \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3})$$

$$< \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3})$$

$$= \frac{1}{1 - (3/16)} cn^2 + \Theta(n^{\log_4 3})$$

$$= \frac{16}{13} cn^2 + \Theta(n^{\log_4 3})$$

$$= O(n^2) \quad (\Theta(n^{\log_4 3}) = O(n^{0,8}) = O(n^2)) .$$

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1-x}$$

(série geométrica)

6.5 MÉTODO DA ÁRVORE DE RECURSÃO

- Podemos usar o método de substituição para verificar que nosso palpite é correto, isto é, $T(n) = O(n^2)$ é um limite superior para a recorrência $T(n) = 3T(n/4) + \Theta(n^2)$. Queremos mostrar que $T(n) \leq dn^2$ para alguma constante $d > 0$. Usando a mesma constante $c > 0$ de antes, temos:

$$T(n) \leq 3T\left(\frac{n}{4}\right) + cn^2 \leq 3d\left(\frac{n}{4}\right)^2 + cn^2 = \frac{3}{16}dn^2 + cn^2 \leq dn^2$$

- em que a última etapa é válida desde que seja escolhido um $d \geq (16/13)c$.

6.5 MÉTODO DA ÁRVORE DE RECURSÃO

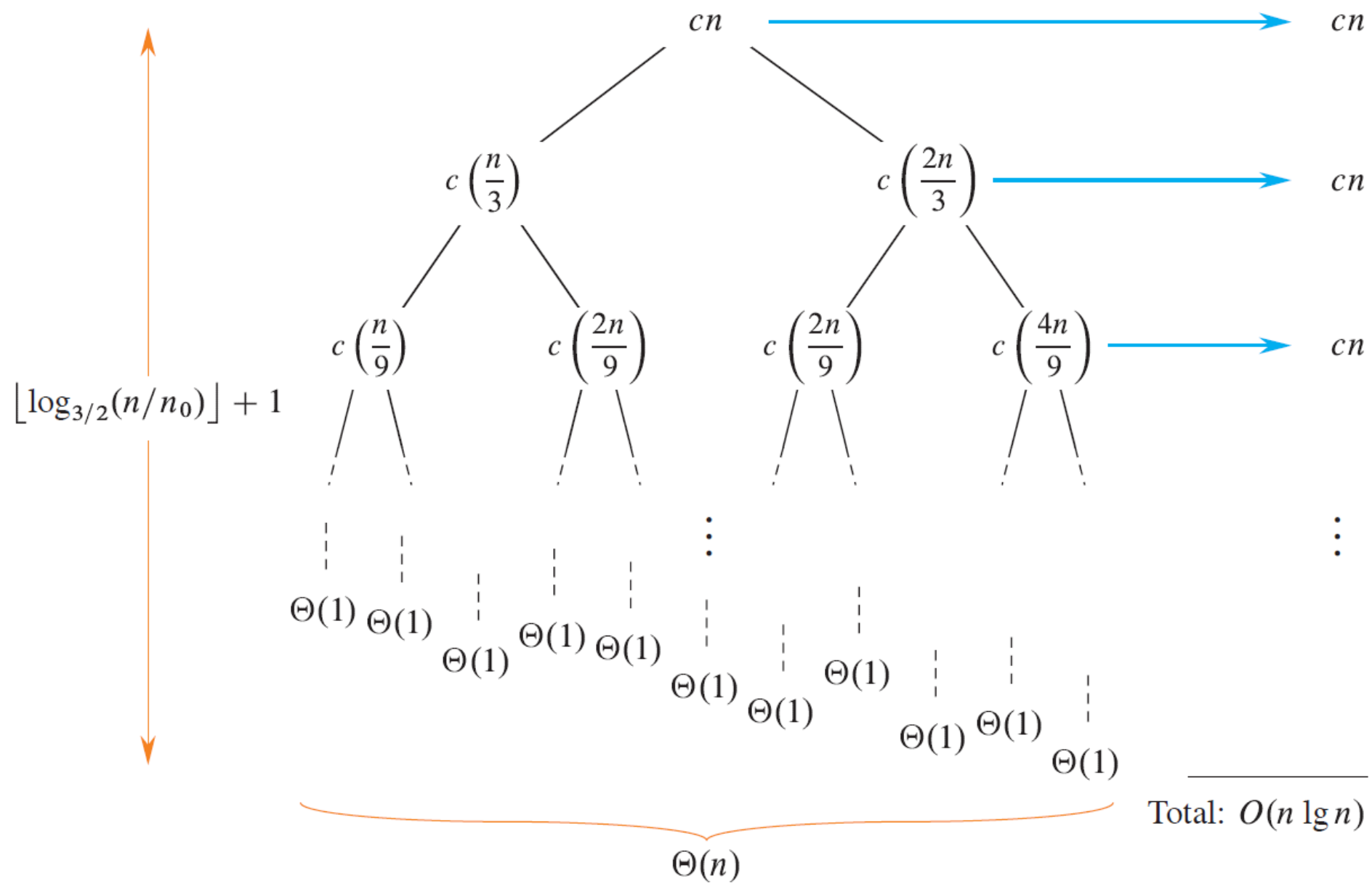
- Para o caso-base da indução, seja $n_0 > 0$ uma constante de limiar suficientemente grande para que a recorrência seja bem definida quando $T(n) = \Theta(1)$ para $n < n_0$.
- Podemos escolher um valor para d grande o suficiente para que d domine a constante oculta por Θ , caso em que $dn^2 \geq T(n)$ para $1 \leq n < n_0$, completando a prova do caso-base.

6.5 MÉTODO DA ÁRVORE DE RECURSÃO

- Exemplo irregular:

$$T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + \theta(n)$$

- Essa árvore de recursão não será balanceada, com diferentes caminhos entre a raiz e as folhas tendo diferentes comprimentos. Partindo da esquerda em qualquer nó, produz-se um subproblema com um terço do tamanho, e partindo para a direita produz-se um subproblema com dois terços do tamanho.



6.5 MÉTODO DA ÁRVORE DE RECURSÃO

- Como antes, c representa o fator constante no termo $\theta(n)$. Quando somamos os valores em todos os níveis da árvore de recursão, obtemos um valor de cn para cada nível. O caminho simples mais longo da raiz até uma folha é $n \rightarrow (2/3)n \rightarrow (2/3)^2n \rightarrow \dots \rightarrow 1$.
- Visto que $(2/3)^k n = 1$ quando $k = \log_{3/2} n$, a altura da árvore é $\log_{3/2} n$.
- Intuitivamente, esperamos que a solução para a recorrência seja, no máximo, o número de níveis vezes o custo de cada nível ou $O(cn \log_{3/2} n) = O(n \lg n)$. Entretanto, a figura anterior mostra apenas os níveis superiores da árvore de recursão, e nem todo nível da árvore contribui com um custo cn .

6.5 MÉTODO DA ÁRVORE DE RECURSÃO

- Considere o custo das folhas. Se essa árvore de recursão fosse uma árvore binária completa de altura $\log_{3/2} n$, haveria $2^{\log_{3/2} n} = n^{\log_{3/2} 2}$ folhas.
- Como o custo de cada folha é uma constante, o custo total de todas as folhas será $\Theta(n^{\log_{3/2} 2})$ que, visto que $\log_{3/2} 2 (\approx 1,7)$ é uma constante estritamente maior do que 1, é $\omega(n \lg n)$.
- Contudo, essa árvore de recursão não é uma árvore binária completa e, por isso, ela tem menos de $n^{\log_{3/2} 2}$ folhas. Além do mais, à medida que descemos em relação à raiz, mais e mais nós internos estão ausentes.

6.5 MÉTODO DA ÁRVORE DE RECURSÃO

- Consequentemente, níveis mais próximos da parte inferior da árvore de recursão contribuem com menos de cn para o custo total.
- Poderíamos desenvolver uma contabilidade precisa de todos os custos, mas estamos apenas tentando propor um palpite para usar no método de substituição. Vamos tolerar o desleixo e tentar mostrar que um palpite $O(n \lg n)$ para o limite superior é correto.
- De fato, podemos usar o método de substituição para verificar que $O(n \lg n)$ é um limite superior para a solução da recorrência. Mostramos que $T(n) \leq dn \lg n$, onde d é uma constante positiva adequada.

6.5 MÉTODO DA ÁRVORE DE RECURSÃO

■ Temos:

$$\begin{aligned}T(n) &\leq T(n/3) + T(2n/3) + cn \\&\leq d(n/3) \lg(n/3) + d(2n/3) \lg(2n/3) + cn \\&= (d(n/3) \lg n - d(n/3) \lg 3) \\&\quad + (d(2n/3) \lg n - d(2n/3) \lg(3/2)) + cn \\&= dn \lg n - d((n/3) \lg 3 + (2n/3) \lg(3/2)) + cn \\&= dn \lg n - d((n/3) \lg 3 + (2n/3) \lg 3 - (2n/3) \lg 2) + cn \\&= dn \lg n - dn(\lg 3 - 2/3) + cn \\&\leq dn \lg n,\end{aligned}$$

desde que $d \geq c/(\lg 3 - (2/3))$.

6.5 MÉTODO DA ÁRVORE DE RECURSÃO

- Exercício: para cada uma das recorrências a seguir, esboce sua árvore de recursão e escolha um bom limite assintótico superior em sua solução. Depois, use o método da substituição para verificar sua resposta.
 - a) $T(n) = T(n/2) + n^3$.
 - b) $T(n) = 4T(n/3) + n$.
 - c) $T(n) = 4T(n/2) + n$.
 - d) $T(n) = 3T(n - 1) + 1$.

6.5 MÉTODO DA ÁRVORE DE RECURSÃO

- $T(n) = T(n/2) + n^3$.

6.5 MÉTODO DA ÁRVORE DE RECURSÃO

■ $T(n) = T(n/2) + n^3$.

Diagram illustrating the recursion tree for $T(n) = T(n/2) + n^3$. The tree has a height of $\lg n + 1$. The work done at each level is shown on the right:

- Level 0: n^3
- Level 1: $\left(\frac{n}{2}\right)^3 \rightarrow \frac{n^3}{8}$
- Level 2: $\left(\frac{n}{4}\right)^3 \rightarrow \frac{n^3}{64}$
- ...

Total work (sum of all levels):

$$\text{Total: } n^3 \sum_{k=0}^{\lg n} \frac{1}{8^k} < n^3 \sum_{k=0}^{\infty} \frac{1}{8^k} = O(n^3)$$

6.5 MÉTODO DA ÁRVORE DE RECURSÃO

■ $T(n) = T(n/2) + n^3$.

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1-x}$$

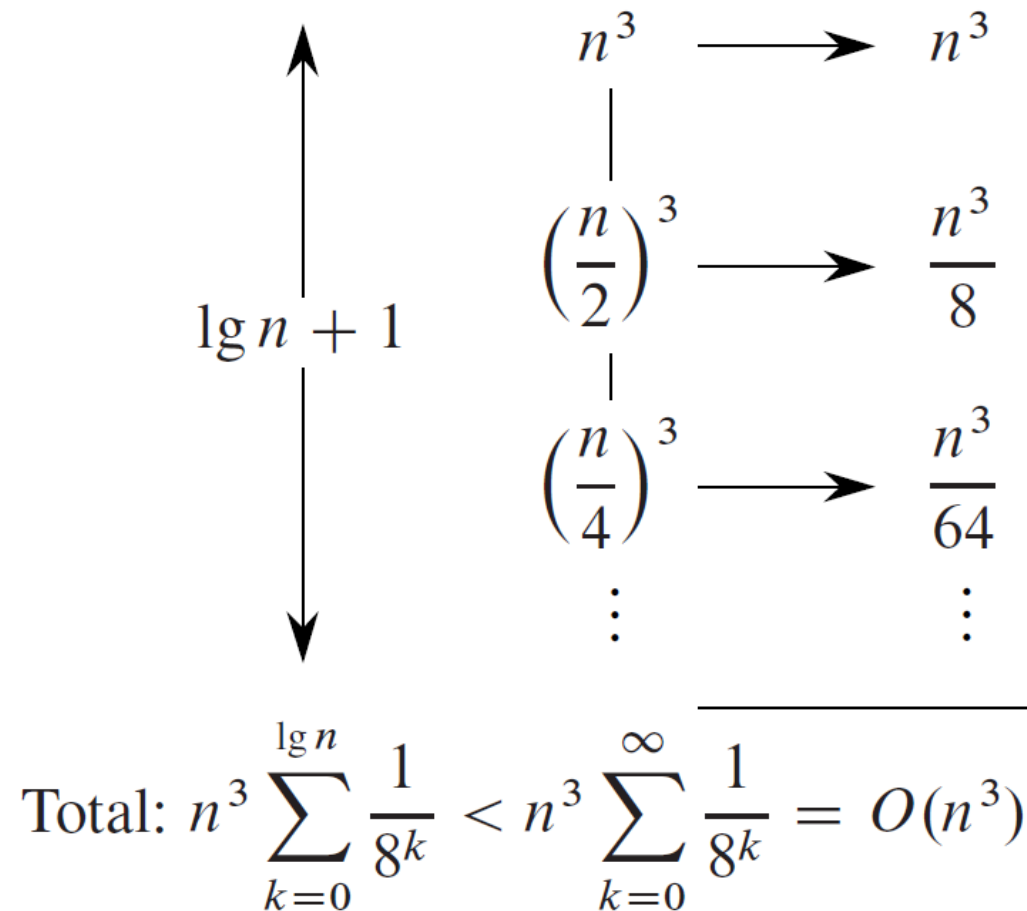


Diagram illustrating the recursion tree for $T(n) = T(n/2) + n^3$. The tree has a height of $\lg n + 1$. The work done at each level is n^3 . The total work is the sum of the work at each level, which is n^3 times the sum of $\frac{1}{8^k}$ from $k=0$ to $\lg n$. This is bounded by n^3 times the sum of $\frac{1}{8^k}$ from $k=0$ to ∞ , which is $O(n^3)$.

$$\text{Total: } n^3 \sum_{k=0}^{\lg n} \frac{1}{8^k} < n^3 \sum_{k=0}^{\infty} \frac{1}{8^k} = O(n^3)$$

6.5 MÉTODO DA ÁRVORE DE RECURSÃO

■ $T(n) = T(n/2) + n^3.$

$$\begin{aligned}\sum_{k=0}^{\lg n} \frac{n^3}{8^k} &< n^3 \sum_{k=0}^{\infty} (1/8)^k \\ &= n^3 \cdot \frac{1}{1 - 1/8} \\ &= (8/7)n^3 \\ &= O(n^3) .\end{aligned}$$

Diagram illustrating the recursion tree for $T(n) = T(n/2) + n^3$. The tree has height $\lg n + 1$. The work at each level is shown as a sequence of terms:

- Level 0: n^3
- Level 1: $\left(\frac{n}{2}\right)^3$
- Level 2: $\left(\frac{n}{4}\right)^3$
- Level 3: $\vdots$

The total work is the sum of work at all levels:

$$\text{Total: } n^3 \sum_{k=0}^{\lg n} \frac{1}{8^k} < n^3 \sum_{k=0}^{\infty} \frac{1}{8^k} = O(n^3)$$

6.5 MÉTODO DA ÁRVORE DE RECURSÃO

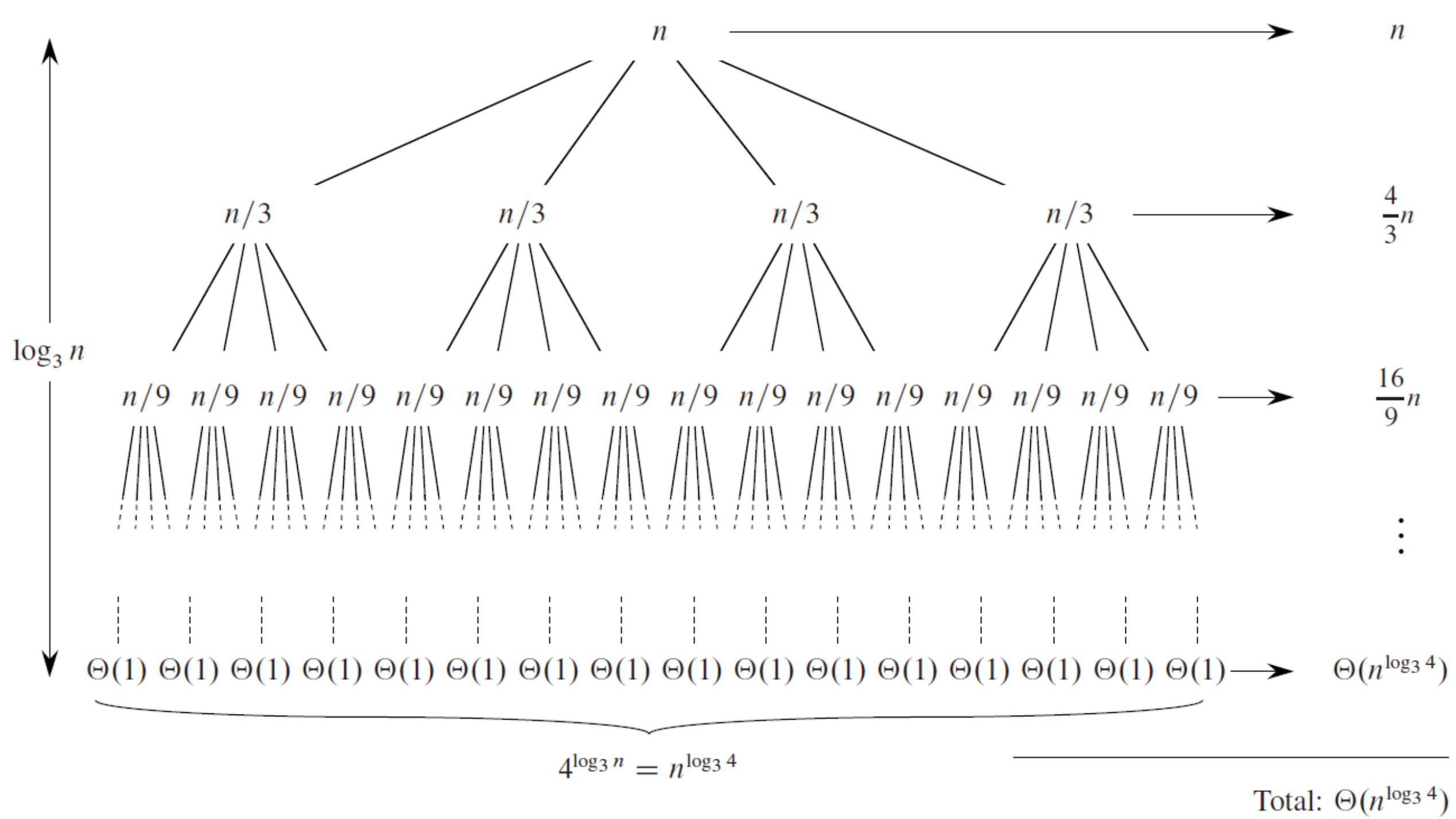
- $T(n) = T(n/2) + n^3$.

$$\begin{aligned}T(n) &= T(n/2) + n^3 \\&\leq c(n/2)^3 + n^3 \\&= cn^3/8 + n^3 \\&= n^3(c/8 + 1),\end{aligned}$$

que é menor ou igual a cn^3 se $c \geq 8/7$.

6.5 MÉTODO DA ÁRVORE DE RECURSÃO

- $T(n) = 4T(n/3) + n.$



6.5 MÉTODO DA ÁRVORE DE RECURSÃO

■ $T(n) = 4T(n/3) + n.$

$$\begin{aligned} \sum_{k=0}^{\log_3 n - 1} \left(\frac{4}{3}\right)^k n + \Theta(n^{\log_3 4}) &= n \cdot \frac{(4/3)^{\log_3 n} - 1}{(4/3) - 1} + \Theta(n^{\log_3 4}) \\ &< 3n(4/3)^{\log_3 n} + \Theta(n^{\log_3 4}) \\ &= 3n(4^{\log_3 n})(1/3)^{\log_3 n} + \Theta(n^{\log_3 4}) \\ &= 3n(n^{\log_3 4})(3^{-\log_3 n}) + \Theta(n^{\log_3 4}) \\ &= 3n^{\log_3 4 + 1} n^{-\log_3 3} + \Theta(n^{\log_3 4}) \\ &= 3n^{\log_3 4 + 1} n^{-1} + \Theta(n^{\log_3 4}) \\ &= 3n^{\log_3 4} + \Theta(n^{\log_3 4}) \\ &= \Theta(n^{\log_3 4}). \end{aligned}$$

6.5 MÉTODO DA ÁRVORE DE RECURSÃO

■ $T(n) = 4T(n/3) + n.$

$$\sum_{k=0}^{\log_3 n - 1} \left(\frac{4}{3}\right)^k n + \Theta(n^{\log_3 4}) = n \cdot \frac{(4/3)^{\log_3 n} - 1}{(4/3) - 1} + \Theta(n^{\log_3 4})$$
$$\sum_{k=0}^n x^k = \frac{x^{n+1} - 1}{x - 1}$$
$$\begin{aligned} &< 3n(4/3)^{\log_3 n} + \Theta(n^{\log_3 4}) \\ &= 3n(4^{\log_3 n})(1/3)^{\log_3 n} + \Theta(n^{\log_3 4}) \\ &= 3n(n^{\log_3 4})(3^{-\log_3 n}) + \Theta(n^{\log_3 4}) \\ &= 3n^{\log_3 4 + 1} n^{-\log_3 3} + \Theta(n^{\log_3 4}) \\ &= 3n^{\log_3 4 + 1} n^{-1} + \Theta(n^{\log_3 4}) \\ &= 3n^{\log_3 4} + \Theta(n^{\log_3 4}) \\ &= \Theta(n^{\log_3 4}). \end{aligned}$$

6.5 MÉTODO DA ÁRVORE DE RECURSÃO

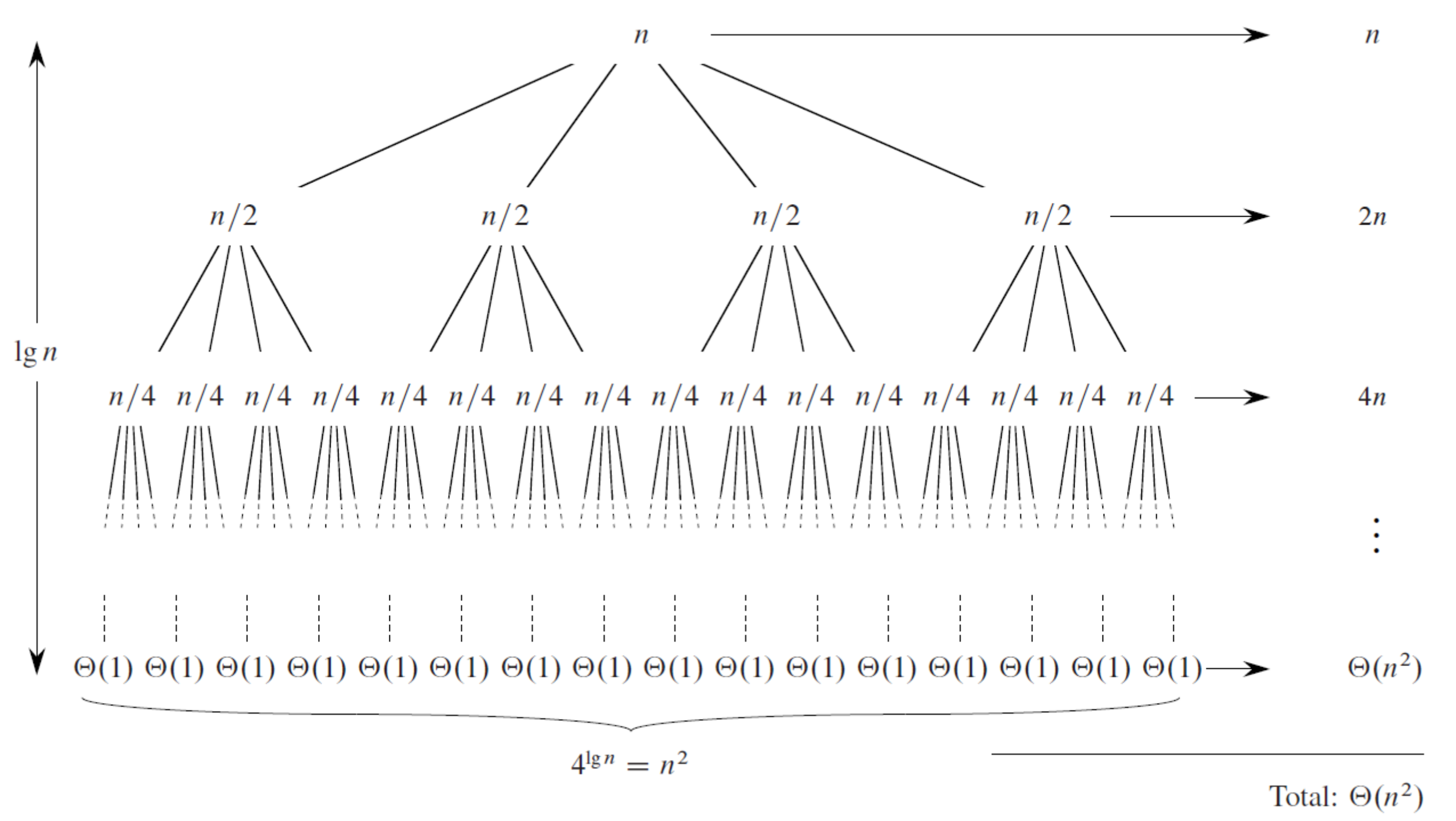
- $T(n) = 4T(n/3) + n$.
- Supomos que $T(n) = n^{\log_3 4} - cn$

$$\begin{aligned}T(n) &= 4T(n/3) + n \\&\leq 4((n/3)^{\log_3 4} - cn/3) + n \\&= 4n^{\log_3 4} (1/3)^{\log_3 4} - (4/3)cn + n \\&= 4n^{\log_3 4} (3^{-\log_3 4}) - (4/3)cn + n \\&= 4n^{\log_3 4} (4^{-\log_3 3}) - (4/3)cn + n \\&= 4n^{\log_3 4} (4^{-1}) - (4/3)cn + n \\&\leq n^{\log_3 4} - cn\end{aligned}$$

que é menor ou igual a $cn^{\log_3 4}$
se $-(4/3)cn + n \leq -cn$ ou $c \geq 3$.

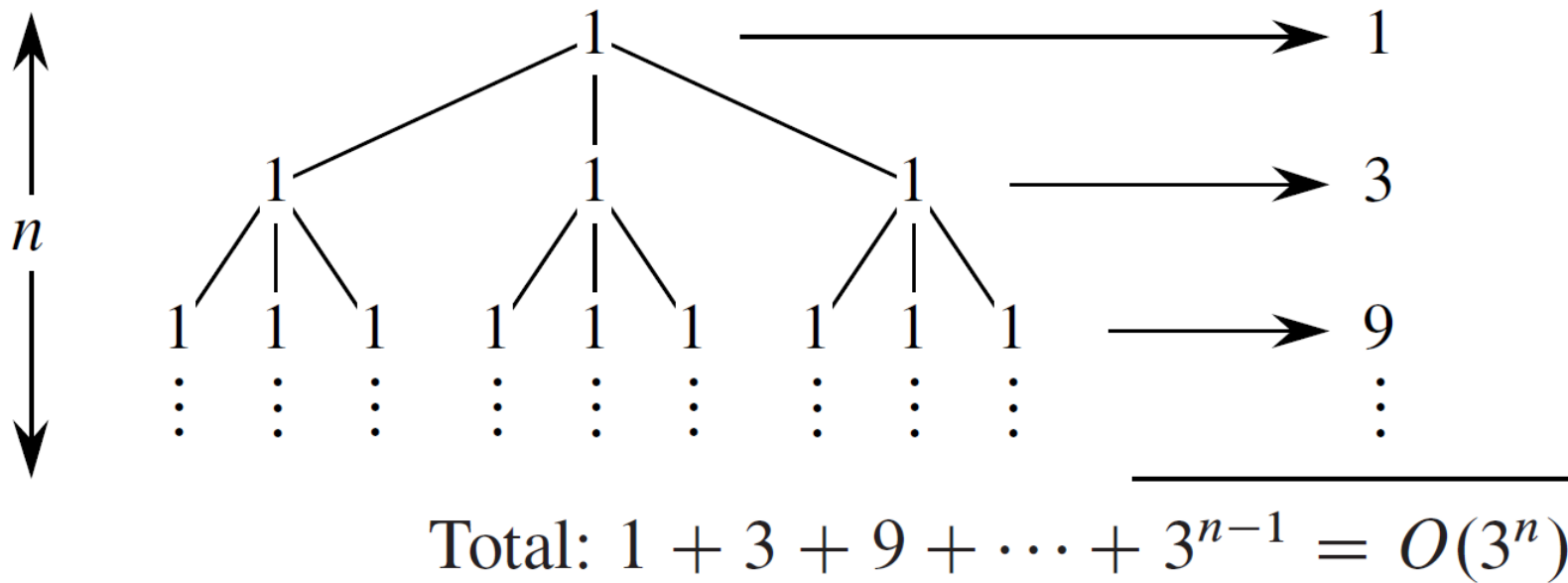
6.5 MÉTODO DA ÁRVORE DE RECURSÃO

- $T(n) = 4T(n/2) + n.$



6.5 MÉTODO DA ÁRVORE DE RECURSÃO

■ $T(n) = 3T(n - 1) + 1$.



6.6 MÉTODO MESTRE

- O método mestre fornece uma “receita” para resolver recorrências algorítmicas da forma:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

em que $a > 0$ e $b > 1$ são constantes. Chamamos $f(n)$ de **função condutora**, e chamamos uma recorrência desse formato geral de **recorrência mestre**.

- Para usar o método mestre, é necessário memorizar três casos, mas com a vantagem de resolver muitas recorrências mestres com grande facilidade.

6.6 MÉTODO MESTRE

- A recorrência mestre descreve o tempo de execução de um algoritmo de divisão e conquista que divide um problema de tamanho n em a subproblemas, cada um de tamanho $n/b < n$.
- Os a subproblemas são resolvidos recursivamente pelo algoritmo, cada um no tempo $T(n/b)$.
- A função condutora $f(n)$ abrange o custo de dividir o problema antes da recursão, além do custo de combinar os resultados das soluções recursivas dos subproblemas.

6.6 MÉTODO MESTRE

- Por exemplo, a recorrência que surge do algoritmo de ordenação por intercalação é uma recorrência mestre com $a = 2$, $b = 2$, e função condutora $f(n) = \Theta(n)$.
- O método mestre permite declarar uma recorrência mestre sem pisos e tetos e deduzi-los implicitamente. Não importa, para o inteiro mais próximo, como os argumentos são arredondados para cima ou para baixo, **os limites assintóticos** que ele fornece **continuam sendo os mesmos**.

6.6 MÉTODO MESTRE

- O método mestre depende do teorema mestre:
- Teorema mestre: sejam $a > 0$ e $b > 1$ constantes, seja $f(n)$ uma função condutora que é definida e não negativa sobre todos os valores reais suficientemente grandes.
- Defina a recorrência $T(n)$ em $n \in \mathbb{N}$ por:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

em que $aT(n/b)$, na realidade, significa $a'T(\lfloor n/b \rfloor) + a''T(\lceil n/b \rceil)$ para algumas constantes $a' \geq 0$ e $a'' \geq 0$ satisfazendo $a = a' + a''$.

6.6 MÉTODO MESTRE

- Então, o comportamento assintótico de $T(n)$ pode ser caracterizado da seguinte forma:
 1. Se existe uma constante $\epsilon > 0$ tal que $f(n) = O(n^{\log_b a - \epsilon})$, então $T(n) = \Theta(n^{\log_b a})$.
 2. Se existe uma constante $k \geq 0$ tal que $f(n) = \Theta(n^{\log_b a} \lg^k n)$, então $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$.
 3. Se existe uma constante $\epsilon > 0$ tal que $f(n) = \Omega(n^{\log_b a - \epsilon})$, e se $f(n)$ além disso satisfizer à condição de regularidade $af(n/b) \leq cf(n)$ para alguma constante $c < 1$ e todos os n suficientemente grandes, então $T(n) = \Theta(f(n))$.

6.6 MÉTODO MESTRE

- A função $n^{\log_b a}$ é chamada de divisora de águas.
- Em cada um dos três casos, comparamos a função condutora $f(n)$ com a função divisora de águas.
- Intuitivamente, se a função divisora de águas crescer assintoticamente mais rápido do que a função condutora, então o caso 1 se aplica. O caso 2 se aplica se as duas funções crescerem praticamente na mesma taxa assintótica. O caso 3 é o “oposto” do caso 1, em que a função condutora cresce assintoticamente mais rápido do que a função divisora de águas. Mas é preciso estar ciente de alguns detalhes técnicos.

6.6 MÉTODO MESTRE

- No caso 1, não somente a função divisora de águas cresce assintoticamente mais rápido do que a função condutora, mas ela deve crescer polinomialmente mais rápido.
- Isto é, a função divisora de águas deve ser assintoticamente maior que a função condutora $f(n)$ por um fator de pelo menos $\Theta(n^\epsilon)$ para alguma constante $\epsilon > 0$. O teorema mestre diz então que a solução é $T(n) = \Theta(n^{\log_b a})$.
- Nesse caso, se examinarmos a árvore de recursão para a recorrência, o custo por nível cresce pelo menos geometricamente da raiz às folhas, e o custo total das folhas é superior ao custo total dos nós internos.

6.6 MÉTODO MESTRE

- No caso 2, as funções divisora de águas e condutora crescem quase na mesma taxa assintótica. Porém, mais especificamente, a função condutora cresce mais rápido que a função divisora de águas por um fator de $\Theta(\lg^k n)$, em que $k \geq 0$.
- O teorema mestre diz que acrescentamos um fator $\lg n$ extra a $f(n)$, resultando na solução $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$. Nesse caso, cada nível da árvore de recursão custa aproximadamente o mesmo — $\Theta(n^{\log_b a} \lg^k n)$ — e existem $\Theta(\lg n)$ níveis.

6.6 MÉTODO MESTRE

- Na prática, a situação mais comum para o caso 2 ocorre quando $k = 0$, caso em que as funções divisoras de água e condutora têm o mesmo crescimento assintótico, e a solução é $T(n) = \Theta(n^{\log_b a} \lg n)$.
- O caso 3 espelha o caso 1. Não apenas a função condutora deve crescer assintoticamente mais rápido que a função divisora de águas, mas ela deve crescer polinomialmente mais rápido. Isto é, a função condutora $f(n)$ deve ser assintoticamente maior que a função divisora de águas por pelo menos um fator de $\Theta(n^\epsilon)$ para alguma constante $\epsilon > 0$. Além disso, a função condutora deve satisfazer à condição de regularidade de que $af(n/b) \leq cf(n)$.

6.6 MÉTODO MESTRE

- Para usarmos o método mestre, simplesmente determinamos qual caso (se houver algum) do teorema mestre se aplica e anotamos a resposta.
- Considere a recorrência $T(n) = 9T(n/3) + n$. Para essa recorrência, temos $a = 9$, $b = 3$, implicando que $n^{\log_b a} = n^{\log_3 9} = \theta(n^2)$.
- Visto que $f(n) = n = O(n^{2 - \epsilon})$, para qualquer constante $\epsilon \leq 1$, podemos aplicar o caso 1 do teorema mestre e concluir que a solução é $T(n) = \Theta(n^2)$.

6.6 MÉTODO MESTRE

- Agora, considere a recorrência $T(n) = T(2n/3) + 1$, na qual $a = 1$ e $b = 3/2$, o que significa que a função divisora de águas é $n^{\log_b a} = n^{\log_{3/2} 1} = n^0 = 1$. Aplica-se o caso 2, já que $f(n) = 1 = \Theta(\lg^0 n) = \Theta(1)$ e, assim, a solução para a recorrência é $T(n) = \Theta(\lg n)$.

6.6 MÉTODO MESTRE

- Para a recorrência $T(n) = 3T(n/4) + n \lg n$, temos $a = 3$ e $b = 4$, o que significa que $n^{\log_b a} = n^{\log_4 3} = O(n^{0,793})$. Visto que $f(n) = n \lg n = \Omega(n^{\log_4 3 + \epsilon})$, em que ϵ pode ser tão grande quanto aproximadamente 0,2, o caso 3 se aplica desde que a condição de regularidade seja mantida para $f(n)$.
- Isso acontece porque, para n suficientemente grande, temos que $af(n/b) = 3(n/4) \lg(n/4) \leq (3/4)n \lg n = cf(n)$ para $c = 3/4$. Pelo caso 3, a solução para a recorrência é $T(n) = \Theta(n \lg n)$.

6.6 MÉTODO MESTRE

- Em seguida, vejamos a recorrência $T(n) = 2T(n/2) + n \lg n$, em que temos $a = 2$, $b = 2$ e $n^{\log_b a} = n^{\log_2 2} = n$. O caso 2 se aplica porque $f(n) = n \lg n = \Theta(\lg^1 n)$. Concluimos que a solução é $T(n) = \Theta(n \lg^2 n)$.
- A $T(n) = 2T(n/2) + \Theta(n)$ caracteriza o tempo de execução da ordenação por intercalação. Visto que $a = 2$ e $b = 2$, a função divisora de águas é $n^{\log_b a} = n^{\log_2 2} = n$. O caso 2 se aplica porque $f(n) = \Theta(n)$, e a solução é $T(n) = \Theta(n \lg n)$.

6.6 MÉTODO MESTRE

- Há situações em que não é possível usar o teorema mestre. Por exemplo, pode ser que a função divisora de águas e a função condutora não possam ser comparadas assintoticamente.
- Podemos ter que $f(n) \gg n^{\log_b a}$ para um número infinito de valores de n , mas também que $f(n) \ll n^{\log_b a}$ para um número infinito de valores diferentes de n .

6.6 MÉTODO MESTRE

- Como questão prática, no entanto, a maioria das funções condutoras que surgem no estudo de algoritmos pode ser significativamente comparada com a função divisora de águas. Se encontrarmos uma recorrência mestre para a qual isso não acontece, teremos que recorrer à substituição ou outros métodos.
- Como exemplo de função condutora caindo em uma lacuna, considere a recorrência $T(n) = 2T(n/2) + n/\lg n$. Como $a = 2$ e $b = 2$, a função divisora de águas é $n^{\log_b a} = n^{\log_2 2} = n^1 = n$.

6.6 MÉTODO MESTRE

- A função condutora é $n/\lg n = o(n)$, o que significa que ela cresce assintoticamente mais devagar do que a função divisora de águas n . Mas $n/\lg n$ cresce apenas logaritmicamente mais lento que n , e não polinomialmente mais lento. Mais precisamente, $\lg n = o(n^\epsilon)$ para qualquer constante $\epsilon > 0$, o que significa que $1/\lg n = \omega(n^{-\epsilon})$ e $n/\lg n = \omega(n^{1-\epsilon}) = \omega(n^{\log_b a - \epsilon})$.
- Assim, nenhuma constante $\epsilon > 0$ existe tal que $n/\lg n = O(n^{\log_b a - \epsilon})$, que é necessário para o caso 1 se aplicar. O caso 2 também não se aplica, pois $n/\lg n = \Theta(n^{\log_b a} \lg^k n)$, em que $k = -1$, mas k deve ser não negativo para que o caso 2 se aplique.

6.6 MÉTODO MESTRE

- Exercício: use o método mestre para fornecer limites assintóticos restritos às recorrências a seguir.
 - a) $T(n) = 2T(n/4) + 1$
 - b) $T(n) = 2T(n/4) + \sqrt{n}$
 - c) $T(n) = 2T(n/4) + \sqrt{n} \lg^2 n$
 - d) $T(n) = 2T(n/4) + n$
 - e) $T(n) = 2T(n/4) + n^2$

6.6 MÉTODO MESTRE

- Exercício: use o método mestre para fornecer limites assintóticos restritos às recorrências a seguir.

a) $T(n) = 2T(n/4) + 1$

b) $T(n) = 2T(n/4) + \sqrt{n}$

c) $T(n) = 2T(n/4) + \sqrt{n} \lg^2 n$

d) $T(n) = 2T(n/4) + n$

e) $T(n) = 2T(n/4) + n^2$

$a = 2, b = 4$, então:

$$n^{\log_b a} = n^{\log_4 2} = n^{\frac{1}{2}} = \sqrt{n}$$

a)

$$T(n) = \Theta(\sqrt{n})$$

$f(n) = O(n^{\frac{1}{2} - \epsilon})$ para $\epsilon = \frac{1}{2}$.

O caso 1 se aplica, e então

$$T(n) = \theta\left(n^{\frac{1}{2}}\right) = \theta(\sqrt{n})$$

6.6 MÉTODO MESTRE

- Exercício: use o método mestre para fornecer limites assintóticos restritos às recorrências a seguir.

a) $T(n) = 2T(n/4) + 1$

b) $T(n) = 2T(n/4) + \sqrt{n}$

c) $T(n) = 2T(n/4) + \sqrt{n} \lg^2 n$

d) $T(n) = 2T(n/4) + n$

e) $T(n) = 2T(n/4) + n^2$

$a = 2, b = 4$, então:

$$n^{\log_b a} = n^{\log_4 2} = n^{\frac{1}{2}} = \sqrt{n}$$

b)

$$T(n) = \Theta(\sqrt{n} \lg n)$$

$$f(n) = \sqrt{n} = \Theta(n^{\log_b a})$$

O caso 2 se aplica, com $k = 0$

6.6 MÉTODO MESTRE

- Exercício: use o método mestre para fornecer limites assintóticos restritos às recorrências a seguir.

a) $T(n) = 2T(n/4) + 1$

b) $T(n) = 2T(n/4) + \sqrt{n}$

c) $T(n) = 2T(n/4) + \sqrt{n} \lg^2 n$

d) $T(n) = 2T(n/4) + n$

e) $T(n) = 2T(n/4) + n^2$

$a = 2, b = 4$, então:

$$n^{\log_b a} = n^{\log_4 2} = n^{\frac{1}{2}} = \sqrt{n}$$

c)

$$T(n) = \Theta(\sqrt{n} \lg^3 n)$$

$$f(n) = \sqrt{n} \lg^2 n = \Theta(n^{\log_b a} \lg^2 n)$$

O caso 2 se aplica, com $k = 2$

6.6 MÉTODO MESTRE

- Exercício: use o método mestre para fornecer limites assintóticos restritos às recorrências a seguir.

a) $T(n) = 2T(n/4) + 1$

b) $T(n) = 2T(n/4) + \sqrt{n}$

c) $T(n) = 2T(n/4) + \sqrt{n} \lg^2 n$

d) $T(n) = 2T(n/4) + n$

e) $T(n) = 2T(n/4) + n^2$

$a = 2, b = 4$, então:

$$n^{\log_b a} = n^{\log_4 2} = n^{\frac{1}{2}} = \sqrt{n}$$

d) e e): Aplicar o caso 3, verificando a condição de regularidade